#SystemDesign

"Great systems are built, not guessed."

COMPUTER ARCHITECTURE

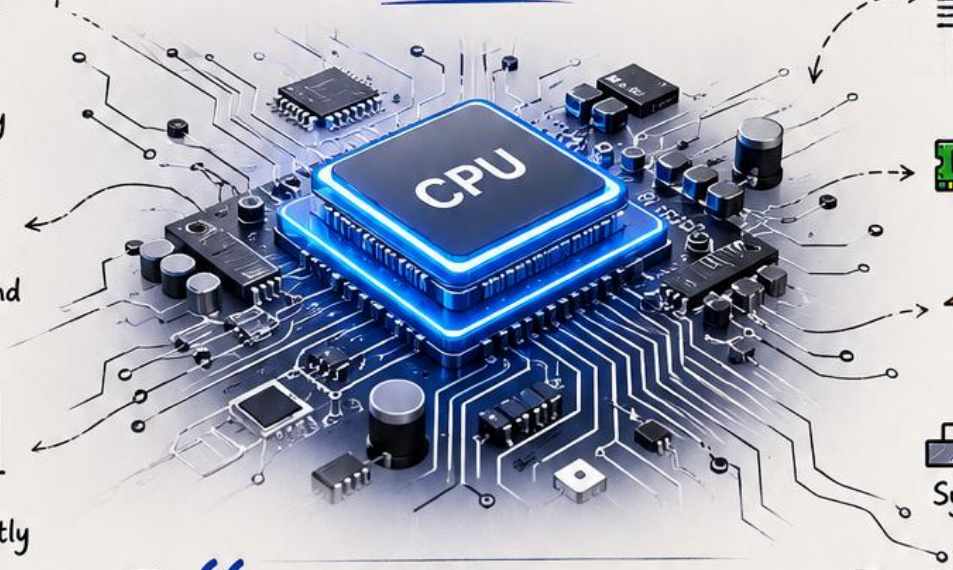
Notes

Learn • Understand • Implement

Design Smartly

Understand Deeply

Build Confidently



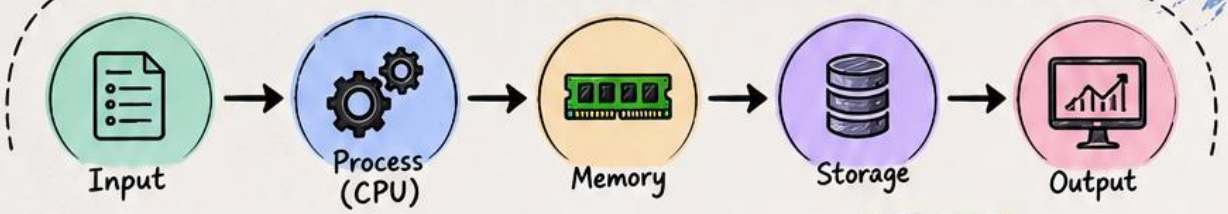
CPU

Memory

I/O

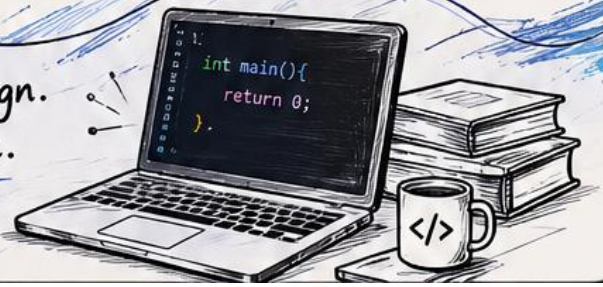
System Bus

> "Architecture is invisible, but its impact is everywhere."



- ✓ Clear Concepts
- ✓ Well Structured
- ✓ Exam & Interview Ready

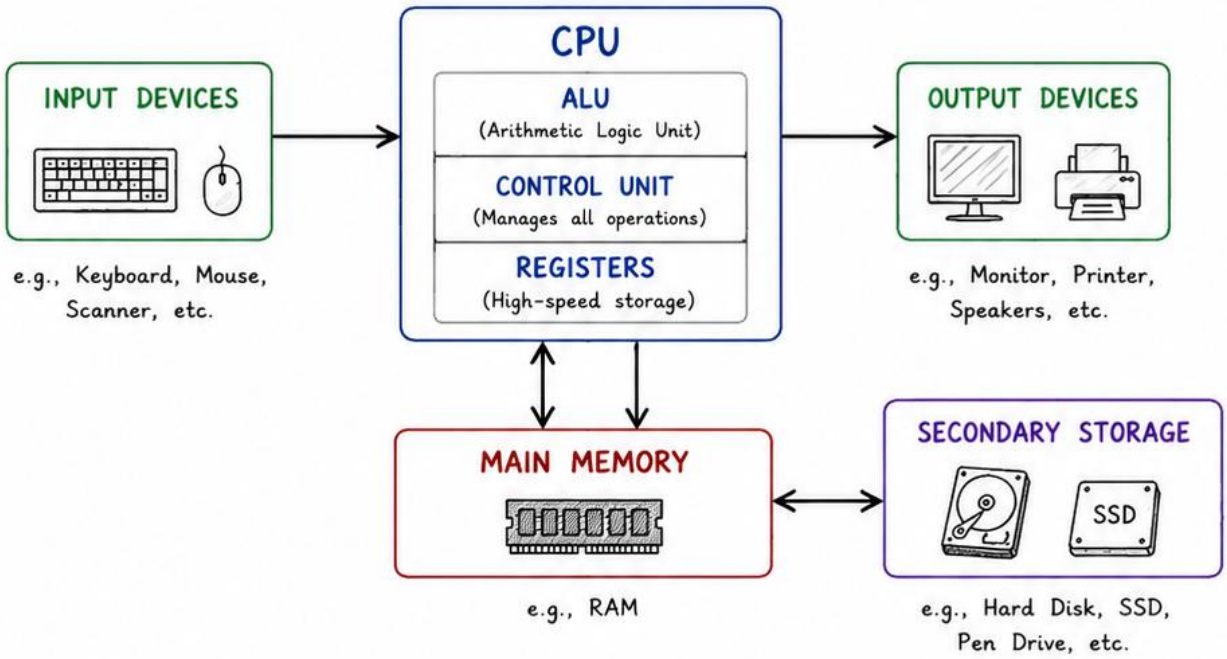
Think. Design. Optimize.



COMPUTER ARCHITECTURE

Chapter 1 : Introduction

Computer Architecture is the foundation of a computer system. It deals with the **design, structure, and organization** of the computer and the ways its components **work together** to perform tasks efficiently.



WHAT WE WILL LEARN

- ✓ CPU, ALU, Control Unit, and Registers
- ✓ Memory and Storage Organization
- ✓ System Bus and Communication
- ✓ Instruction Cycle and Execution
- ✓ Performance, Pipelining & Parallel Processing
- ✓ Modern Architectures and more

KEY TAKEAWAYS

- ★ Design of a computer system
- ★ How data is processed inside
- ★ How instructions are executed
- ★ How components communicate
- ★ How performance is improved

FINAL TAKEAWAY

Computer Architecture helps us understand how a computer works internally and how all its parts work together to complete any task accurately and efficiently.

2. BASIC COMPUTER ORGANIZATION

A computer is organized into major functional units that work together to accept data, process it, store it, and produce output.

INPUT UNIT



Accepts data and instructions from the user or other devices and converts them into machine readable form.

CPU (CENTRAL PROCESSING UNIT)

ALU (Arithmetic Logic Unit)
Performs arithmetic operations (+, −, ×, ÷) and logical operations (AND, OR, NOT, etc.).

CONTROL UNIT (CU)
Controls and coordinates all the operations in the computer. It manages the flow of data and instructions.

REGISTERS
Small, high-speed storage locations inside the CPU used to store data, instructions and intermediate results.

OUTPUT UNIT



Takes the processed information from the computer and presents it to the user in a usable form.

MEMORY UNIT

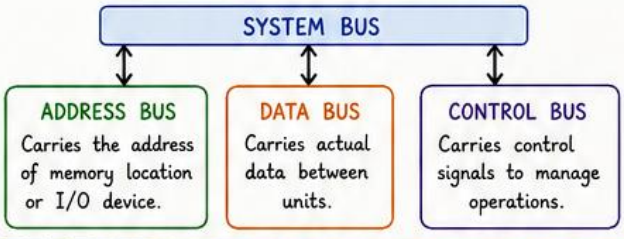
PRIMARY MEMORY (Main Memory)	SECONDARY MEMORY (Auxiliary Storage)
• RAM	• Hard Disk
• ROM	• SSD
• Cache	• Pen Drive, etc.

I/O INTERFACE

Acts as a bridge between I/O devices and the rest of the system.

SYSTEM BUS

The system bus is the communication pathway that transfers data, address and control signals between the CPU, memory and I/O devices.



HOW IT WORKS

1. Input unit accepts data / instructions.
2. Data is stored in memory.
3. CPU fetches instructions from memory.
4. CPU decodes and executes the instructions.
5. Results are stored in memory or sent to output unit.
6. Output unit presents the final result.

KEY POINTS

- ★ All units are interconnected and work as one system.
- ★ CPU is the brain of the computer.
- ★ Memory stores data, instructions and results.
- ★ I/O units allow communication with outside world.
- ★ System bus ensures smooth data transfer among all units.



KEY TAKEAWAY

Basic computer organization shows how all the major components work together to accept input, process it, store it and produce output efficiently.



3. CPU, ALU, CU & REGISTERS

The CPU (Central Processing Unit) is the brain of the computer.
It performs all the processing and controls the operations of all units.

CPU = ALU + CU + REGISTERS

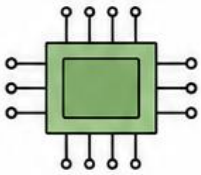
CPU

ALU
(Arithmetic Logic Unit)



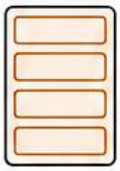
- Performs arithmetic operations: +, −, ×, ÷
- Performs logical operations: AND, OR, NOT, XOR, etc.
- Works on data and produces result.

CU
(Control Unit)



- Controls and coordinates all operations in the CPU.
- Fetches instructions from memory.
- Decodes instructions.
- Sends control signals to ALU, Registers, Memory, and I/O units.

REGISTERS
(High Speed Storage)



- Small, very fast storage locations inside CPU.
- Store data, addresses, instructions and intermediate results.
- Types: General Purpose Registers, Special Purpose Registers.

HOW THEY WORK TOGETHER

1 FETCH
CU fetches instruction from memory and stores it in a register (e.g., IR).

→

2 DECODE
CU decodes the instruction to understand what operation is required.

→

3 EXECUTE
CU sends control signals to ALU. ALU performs the operation on data from registers.

→

4 STORE
Result is stored back in register or memory. Cycle repeats for next instruction.

TYPES OF REGISTERS

- **General Purpose Registers** – Used to store data temporarily during processing.
- **Special Purpose Registers** – Have specific functions in CPU.
  - ➢ PC (Program Counter) – Holds address of next instruction.
  - ➢ IR (Instruction Register) – Holds current instruction.
  - ➢ MAR (Memory Address Register) – Holds address of memory location.
  - ➢ MDR (Memory Data Register) – Holds data read from or written to memory.

KEY POINTS

- ★ ALU performs calculations and logic operations.
- ★ CU controls and coordinates all activities.
- ★ Registers provide high-speed storage inside CPU.
- ★ These three units work together to execute instructions efficiently.
- ★ CPU is the heart of the computer system.



TAKEAWAY

ALU does the work, CU controls the work, and Registers hold the data and instructions needed for the work.

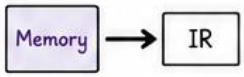


4. INSTRUCTION CYCLE

The Instruction Cycle is the process by which the CPU fetches an instruction from memory, decodes it, executes it and stores the result. This cycle repeats for every instruction in a program.

STEPS IN INSTRUCTION CYCLE

1 FETCH



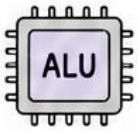
- CPU uses the address in the Program Counter (PC) to fetch the next instruction from memory.
- The instruction is loaded into the Instruction Register (IR).
- PC is updated to point to the next instruction.

2 DECODE



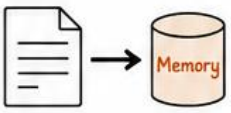
- Control Unit (CU) decodes the instruction in IR.
- It determines the operation to be performed and the operands (data) needed.
- CU generates control signals for the next step.

3 EXECUTE



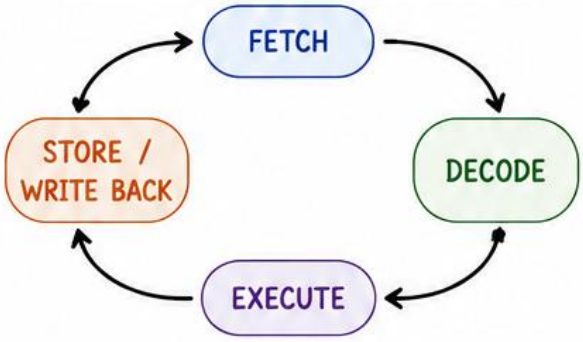
- ALU performs the required operation (arithmetic / logical).
- Operands are taken from registers or memory.
- Result is stored in a register or memory.

4 STORE / WRITE BACK



- Result of execution is written back to the destination register or memory.
- Status flags (Zero, Carry, Sign, etc.) may also be updated.
- Cycle repeats for the next instruction.

FLOW OF INSTRUCTION CYCLE



IMPORTANT POINTS

- ★ The number of clock cycles required for one instruction depends on the type of instruction and the system.
- ★ Some instructions may require extra cycles for fetching operands from memory.
- ★ Instruction cycle is the basic operation performed by CPU.
- ★ Faster execution depends on reducing the time taken in each stage.

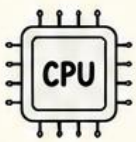
ROLES OF CPU COMPONENTS IN INSTRUCTION CYCLE

Component	Role in Instruction Cycle	Example
Program Counter (PC)	Holds address of next instruction	PC = 2050H
Instruction Register (IR)	Holds the current instruction	IR = 8 3 4 1 (Example instruction)
Control Unit (CU)	Decodes instruction and issues control signals	Generate signals for ALU, Memory, I/O
ALU	Executes the operation	Performs ADD, SUB, AND, OR, etc.
Registers	Store data, addresses, intermediate results	Acc, R1, R2, etc.
Memory	Stores instructions and data	Stores program and variables



KEY TAKEAWAY

Instruction Cycle is the heart of CPU operation.
It ensures that each instruction is fetched, decoded, executed and stored before moving to the next one.



5. INSTRUCTION FORMATS & TYPES

An instruction is a command given to the CPU to perform a specific operation. It is stored in memory and consists of two main parts: OPCODE (operation code) and OPERAND(S) (data or address).

INSTRUCTION FORMAT

The layout of bits in an instruction is called its format.

A general instruction format is:

OPCODE (Operation Code)	ADDRESS / MODE (If required)	OPERAND / DATA (If required)

Total Instruction Length (in bits)

★ **OPCODE**
Specifies the operation to be performed.

OPERAND
Data, address or register on which the operation is performed.

TYPES OF INSTRUCTIONS

Instructions are classified based on the operation they perform.

Type	Description	Example	Example Meaning
1. Data Transfer	Transfer data between registers, memory and I/O.	MOV R1, R2	Copy data from R2 to R1.
2. Arithmetic	Perform arithmetic operations.	ADD R1, R2	R1 ← R1 + R2
3. Logical	Perform logical operations (AND, OR, NOT, XOR, etc.).	AND R1, R2	R1 ← R1 AND R2
4. Control Transfer	Change the sequence of execution.	JMP 2000H	Jump to memory location 2000H.
5. Input / Output	Transfer data between CPU and I/O devices.	IN R1, 80H	Read data from I/O port 80H into R1.
6. Stack	Perform operations on stack (push, pop).	PUSH R1	Push content of R1 onto stack.
7. Machine Control	Control interrupts, system flags, etc.	HLT	Halt the program execution.

COMMON INSTRUCTION FORMATS

Format Type	Structure	Used For
1. Three Address	OPCODE | A1 | A2 | A3	Complex operations, more operands
2. Two Address	OPCODE | A1 | A2	Most general purpose instructions
3. One Address	OPCODE | A	Accumulator based operations
4. Zero Address	OPCODE	Stack based operations

ADDRESSING MODES (BRIEF)

Mode	Meaning	Example
Immediate	Operand is part of the instruction	MOV R1, #25
Direct	Address of operand is given	MOV R1, 2050H
Register	Operand is in a register	MOV R1, R2
Register Indirect	Address is in a register	MOV R1, (R2)
Indexed	Effective address = Base + Index	MOV R1, 1000H(R2)

KEY TAKEAWAY

- ☑ Instruction format defines how an instruction is organized in bits.
- ☑ Instructions are of different types depending on the operation.
- ☑ CPU understands instructions using opcode and operands.
- ☑ Different formats and addressing modes help in efficient programming and execution.

6. ADDRESSING MODES

Addressing Mode is the method used by an instruction to specify the location of its operand.



It tells the CPU how to find the actual data or the effective address required for an operation.

COMMON ADDRESSING MODES

Mode	Description	How it Works	Example	Effective Address (EA)
1. Immediate `#25`	Operand is part of the instruction itself.	The value is used directly as the operand.	MOV R1, #25 (Load 25 into R1)	Not required (Value is in instruction)
2. Direct `2050H`	Address field contains the memory address of the operand.	The address in the instruction points directly to the memory location.	MOV R1, 2050H	EA = 2050H
3. Register `R2`	Operand is in a register.	The instruction specifies the register name.	MOV R1, R2	Not required (Data is in register)
4. Register Indirect `R2` → `5000H`	Register contains the address of the operand.	The address in the register points to the memory location of the operand.	MOV R1, (R2)	EA = (R2) (Content of R2)
5. Indexed `2000H` + `R3`	A base address is added to the content of an index register.	EA is obtained by adding the index register content to the address field.	MOV R1, 2000H(R3)	EA = 2000H + (R3)
6. Base Register `R4` + `1000H`	A base register content is added to the address field.	EA is obtained by adding base register content to the address field.	MOV R1, 1000H(R4)	EA = (R4) + 1000H
7. Relative / PC Relative) `PC` + `20`	Address is obtained by adding a displacement to the Program Counter.	Used in branch/jump instructions (relative addressing).	JMP 20 (Jump ahead by 20)	EA = (PC) + 20
8. Auto Increment `R1` → `(R1)+`	Register points to operand; after access, register is incremented.	After fetching the operand from memory, the register is increased.	MOV R0, (R1)+	EA = (R1) Then R1 = R1 + d
9. Auto Decrement `-(R1)` → `R1`	Register is decremented first, then used to access operand.	Register is decreased first, then the memory location is accessed.	MOV R0, -(R1)	R1 = R1 - d EA = (R1)

EXTRA NOTES

- EA (Effective Address) is the actual memory location where the operand is found.
- Different addressing modes provide flexibility and help reduce instruction size.
- Choosing the right addressing mode can improve program efficiency.

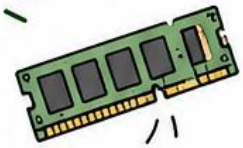
QUICK COMPARISON

- Immediate → Data is in instruction
- Direct → Address is in instruction
- Register → Data is in register
- Register Indirect → Address is in register
- Indexed / Base → Base + Index
- Relative → Based on PC
- Auto Inc/Dec → Register updates automatically

KEY TAKEAWAY

Addressing modes define how an instruction locates its operand.
They play a crucial role in reducing memory access time and instruction length.

7. MEMORY ORGANIZATION

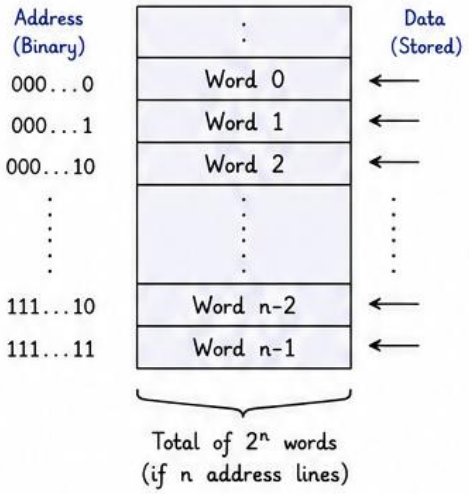


Memory organization refers to how the computer's main memory is structured, addressed and operated to store instructions and data efficiently.

1. BASIC CONCEPT

- Main memory is a large array of storage locations.
- Each location (word) has a unique address.
- CPU can read from or write to any memory location.
- Memory is measured in bytes. (1 Byte = 8 bits)

Memory as an Array of Words



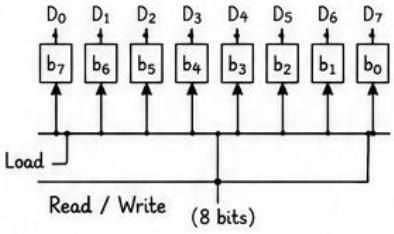
KEY TERMS

- **Word:** Basic unit of data transferred between CPU and memory.
- **Address:** Unique binary number that identifies a memory location.
- **Capacity:** Total number of bytes that can be stored.
- **Access Time:** Time taken to read or write data.
- **Memory Cycle Time:** Minimum time between two consecutive memory accesses.

2. MEMORY UNIT

A memory unit consists of a group of flip-flops (or latches) used to store a word.

Example: 8-bit memory unit



3. MEMORY ADDRESSING

If a memory has 2^n locations, it requires n address lines.

Example:

If memory has 1024 words (2^{10}), then 10 address lines are needed (A0 to A9).

No. of Words	Address Lines (n)
16 (2^4)	4
256 (2^8)	8
1024 (2^{10})	10
4096 (2^{12})	12

4. TYPES OF MEMORY

1. RAM (Random Access Memory)
   - Read/Write memory.
   - Volatile (data lost when power off).
   - Example: DRAM, SRAM.
2. ROM (Read Only Memory)
   - Read only memory.
   - Non-volatile (data retained).
   - Example: PROM, EPROM, EEPROM.
3. Cache Memory
   - Small, fast memory between CPU and main memory.
   - Stores frequently used data.

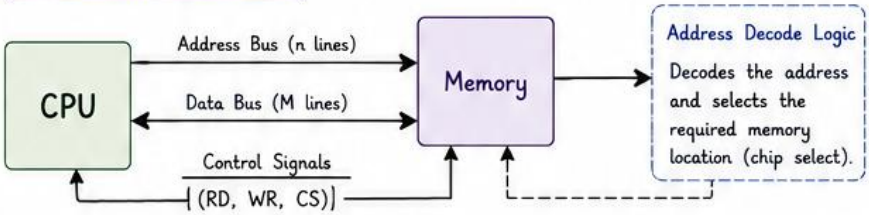
5. MEMORY OPERATION

Operation	Function	Control Signals	Description
Read	Read data from memory	RD = 0, WR = 1	Address is placed on address lines. Data is read from memory to data lines.
Write	Write data to memory	RD = 1, WR = 0	Address is placed on address lines. Data from data lines is written to memory.
Idle	No operation	RD = 1, WR = 1	Memory remains inactive.

IMPORTANT POINTS

- ★ Memory is organized as an array of words.
- ★ Address lines determine which location is being accessed.
- ★ Data lines transfer the actual data.
- ★ Control signals decide Read or Write operation.
- ★ Faster memory with lower access time improves system performance.

6. MEMORY INTERFACE



KEY TAKEAWAY

Memory organization defines how data and instructions are stored, accessed and managed in the computer's main memory system.

8. CACHE MEMORY & MEMORY HIERARCHY

8.1 CACHE MEMORY

Cache memory is a small, very fast memory placed between CPU and main memory. It stores copies of frequently used data/instructions.

PURPOSE

- Reduce average memory access time
- Bridge the speed gap between CPU and main memory
- Improve overall system performance

HOW CACHE WORKS

1. CPU requests data.
2. If data is in cache → Cache Hit (fast access).
3. If data is not in cache → Cache Miss.
   - Required block is fetched from main memory.
   - It is stored in cache (replacing some block if needed).
   - Then data is sent to CPU.

CACHE PERFORMANCE

$$\text{Hit Ratio } (h) = \frac{\text{No. of Hits}}{\text{Total Accesses}} \qquad \text{Miss Ratio } (m) = 1 - h$$

Average Memory Access Time (AMAT)

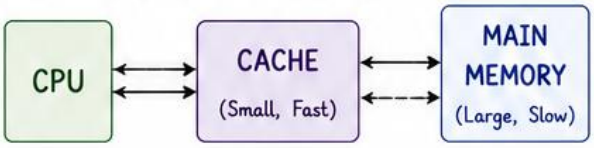
= (h x Hit Time) + (m x Miss Penalty)

Hit Time → Time to access cache

Miss Penalty → Time to fetch from main memory

CACHE ORGANIZATION

A cache is divided into blocks (lines). Each block contains a copy of a main memory block.



TYPES OF CACHE

- L1 Cache: Smallest & fastest, built into CPU.
- L2 Cache: Larger, slightly slower, may be on/off chip.
- L3 Cache: Largest, slower than L1 & L2, shared among cores.

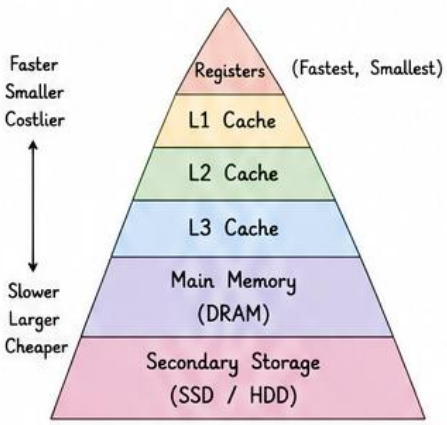
CACHE MAPPING TECHNIQUES

Technique	Description	Example
1. Direct Mapping	A memory block can be placed in only one specific cache line.	Block i → Line (i mod n)
2. Fully Associative	A memory block can be placed in any cache line.	Block i → Any line
3. Set Associative	Cache is divided into sets. Block can be placed in any line of its set.	Block i → Any line in set (i mod s)

★ Tag: Identifies which memory block is stored.
Index: Identifies cache line/set.
Offset: Identifies byte/word within the block.

8.2 MEMORY HIERARCHY

Memory hierarchy is a multi-level structure of memories where each level has different capacity, speed, and cost.



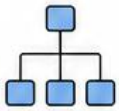
LEVELS OF MEMORY HIERARCHY

Level	Example	Speed	Capacity	Cost per bit	Managed by
1. Registers	CPU Registers	Very High	Very Small	Very High	Compiler / CPU
2. L1 Cache	On-chip	Very High	Small	High	Hardware
3. L2 Cache	On-chip / Off-chip	High	Medium	Medium	Hardware
4. L3 Cache	Off-chip	Medium	Large	Low	Hardware
5. Main Memory	DRAM	Low	Large	Very Low	OS / Hardware
6. Secondary Storage	SSD / HDD	Very Low	Very Large	Extremely Low	OS

PRINCIPLE OF MEMORY HIERARCHY

Keep the most frequently used data in the upper (faster) levels.
Move less frequently used data to lower (slower) levels.





BENEFITS

- ✓ Reduces average memory access time
- ✓ Improves CPU utilization
- ✓ Cost-effective storage
- ✓ Better overall performance



KEY TAKEAWAY

- Cache memory is a small and fast memory that holds frequently used data.
- Cache reduces access time by exploiting locality (temporal & spatial).
- Memory hierarchy balances speed, capacity, and cost for efficient operation.



9. INPUT AND OUTPUT ORGANIZATION

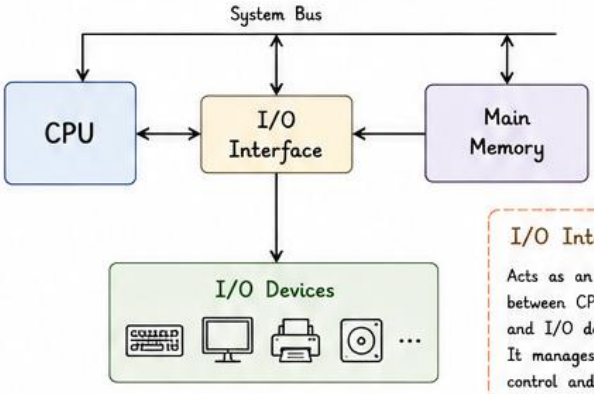
I/O organization deals with how a computer communicates with external devices. It includes the devices, interfaces, data transfer methods and control mechanisms that enable input and output operations.

1. I/O DEVICES

I/O devices allow the computer to interact with the outside world.

Input Devices	Output Devices
• Keyboard	• Monitor
• Mouse	• Printer
• Scanner	• Speaker
• Microphone	• Plotter
• Sensor, etc.	• Actuator, etc.

2. BASIC I/O ORGANIZATION



I/O Interface

Acts as an intermediary between CPU/memory and I/O devices. It manages data, control and timing.

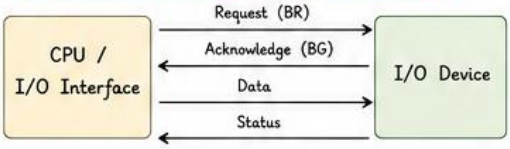
3. I/O INTERFACE (CONTROLLER)

- An interface (or controller) manages the communication between CPU/memory and I/O devices.
- It has:
  - Data Register: Stores data to be sent or received.
  - Status Register: Contains status information of the device.
  - Control Register: Contains control signals and commands.
- It handles timing, handshaking and error detection.

4. DATA TRANSFER METHODS

Method	Description	Advantage	Disadvantage
Programmed I/O	CPU issues I/O command and waits (polls) until the operation completes.	Simple to implement	CPU is idle during I/O
Interrupt Driven I/O	CPU starts I/O and continues executing other tasks. I/O device interrupts CPU when done.	Better CPU utilization	Interrupt overhead
Direct Memory Access (DMA)	Data is transferred directly between I/O device and memory without CPU involvement.	High speed, minimal CPU involvement	More complex hardware

5. HANDSHAKING (SYNCHRONIZATION)

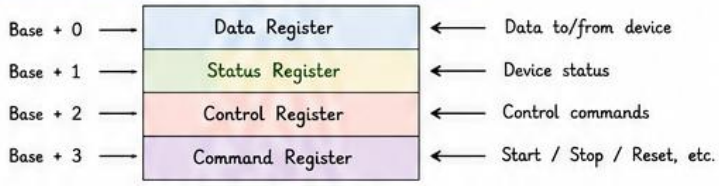


Handshaking ensures proper coordination between CPU and I/O device during data transfer.

6. I/O OPERATIONS

1. CPU issues I/O command by writing to the I/O interface.
2. I/O interface checks device status.
3. Data is transferred using one of the transfer methods.
4. Status is updated and CPU is informed (if required).
5. Data is available in memory or device.

7. I/O INTERFACE STRUCTURE (EXAMPLE)



CPU accesses these registers using memory-mapped I/O or isolated I/O instructions.

8. MEMORY-MAPPED vs I/O-MAPPED I/O

Feature	Memory-Mapped I/O	I/O-Mapped (Isolated) I/O
Address Space	Uses memory address space	Uses separate I/O address space
Instructions Used	Memory read/write	Special I/O instructions (IN/OUT)
Address Lines	All address lines are used	Usually uses fewer address lines
Hardware	Simple	Slightly complex
Flexibility	More flexible	Less flexible

9. KEY POINTS

- ★ I/O devices allow communication between computer and outside world.
- ★ I/O interface controls and manages all I/O operations.
- ★ Data transfer can be Programmed, Interrupt-driven or DMA.
- ★ Handshaking ensures reliable data exchange.
- ★ Memory-mapped I/O is more flexible and widely used.

KEY TAKEAWAY

Efficient I/O organization ensures smooth communication between CPU, memory and I/O devices, improving system performance and reliability.

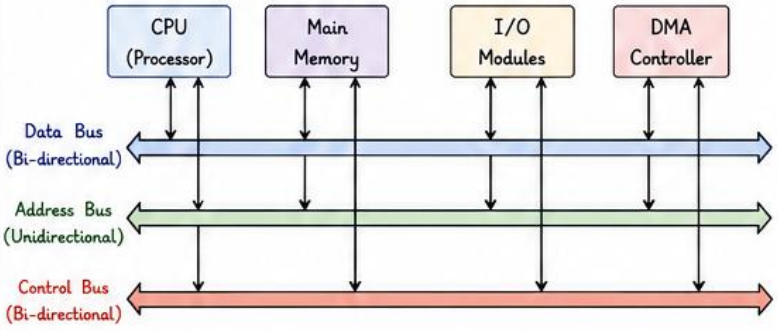
10. SYSTEM BUS & BUS ARCHITECTURE

A system bus is a set of communication lines that transfers data, address and control signals among the CPU, memory and I/O devices.
It acts as the communication pathway inside the computer.

1. WHAT IS SYSTEM BUS?

- The bus is shared communication pathway.
- It connects all major components of a computer system.
- It carries data, addresses and control signals.
- It determines how fast and how efficiently components can communicate.

2. BASIC BUS ARCHITECTURE



3. TYPES OF BUS

Bus Type	Direction	Purpose	Example Signals
Data Bus	Bi-directional	Transfers actual data between CPU, memory and I/O.	D0 – D7, D0 – D31
Address Bus	Unidirectional	Carries address from CPU to memory or I/O device.	A0 – A15, A0 – A31
Control Bus	Bi-directional	Carries control and timing signals.	Read, Write, Memory Select, I/O Select, Interrupt, Clock, Reset

4. BUS STRUCTURE DETAILS

➤ **Data Bus Width (n bits)** : Amount of data transferred at a time.
- Wider bus → more data transfer per clock → higher performance.

➤ **Address Bus Width (m bits)** : Determines maximum memory addresses.
- Maximum addressable memory = 2^m locations.

➤ **Control Bus** : Coordinates all operations on the bus.
- Includes control, timing and status signals.

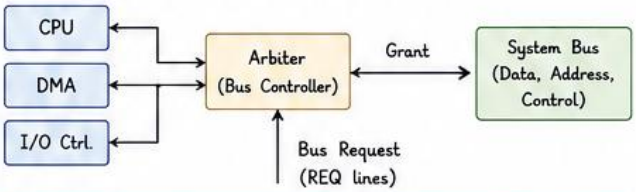
Example:
If address bus = 20 lines → CPU can address 2^{20} locations = 1 MB (when each location is 1 byte).

5. BUS OPERATION (EXAMPLE: MEMORY READ)

	CPU	Bus	Memory
Step 1	Places address on Address Bus	Address →	Receives address
Step 2	Sends Read signal	Read signal →	Prepares data
Step 3	Waits	Data ready ←	Places data on Data Bus
Step 4	Reads data from Data Bus	Data ←	Holds data
Step 5	Removes Read signal	Read inactive →	Ready for next operation

6. BUS ARBITRATION (MULTIPLE BUS MASTERS)

- When multiple devices (e.g., CPU, DMA, I/O controllers) need to use the bus, an arbitration mechanism is used.
- It decides which device gets control of the bus.



Arbiter uses priority or fairness policy to avoid conflicts.

7. BUS CHARACTERISTICS

Characteristic	Description
Bus Width	Number of data/address lines. Wider bus → more data, more addresses.
Bus Speed	How fast data can be transferred.
Bus Bandwidth	Total data transfer capacity (Bandwidth = Width × Speed).
Scalability	Ability to support more devices.
Timing	Synchronization of signals for correct operation.

8. COMMON BUS ARCHITECTURES

Architecture	Description
Single Bus	One bus used for memory and I/O transfers.
Separate Bus	Separate bus for memory and separate bus for I/O.
Hierarchical Bus	High-speed bus near CPU, slower buses for I/O devices.
Point-to-Point	Direct links between components (modern systems).

9. KEY POINTS

- ★ System bus connects CPU, memory and I/O devices.
- ★ It has three main parts: Data Bus, Address Bus, Control Bus.
- ★ Bus width affects data transfer and addressable memory.
- ★ Control signals manage read/write and device operations.
- ★ Arbitration is used when multiple masters need the bus.
- ★ Efficient bus design improves overall system performance.

KEY TAKEAWAY

System bus is the backbone of computer architecture. Well-designed bus architecture ensures fast, reliable and coordinated communication among all components.

11. PIPELINING

Pipelining is a technique used to increase the instruction throughput of a CPU.
It divides the instruction execution into a series of stages.
While one instruction is in one stage, other instructions can be in different stages simultaneously.
Like an assembly line!

1. BASIC IDEA

- In non-pipelined execution, each instruction goes through all stages one by one.
- In pipelined execution, multiple instructions overlap in their execution.
- This increases throughput (instructions completed per unit time).

KEY TERMS

- **Throughput** : Number of instructions completed per unit time.
- **Latency** : Time taken to complete one instruction.
- **Pipeline** : A sequence of stages where each stage performs a part of the instruction execution.

2. INSTRUCTION PIPELINE STAGES (5-STAGE EXAMPLE)

1. IF — Instruction Fetch
2. ID — Instruction Decode & Register Read
3. EX — Execute (ALU)
4. MEM — Memory Access
5. WB — Write Back to Register

IF : Fetch instruction from memory using PC
ID : Decode instruction and read operands from registers
EX : Perform ALU operation / address calculation
MEM : Access memory for load/store instructions
WB : Write result back to register

Different processors may have more or fewer stages, but the idea is the same.

3. NON-PIPELINED VS PIPELINED EXECUTION

(a) Non-Pipelined Execution

Each instruction completes before the next starts.

Instruction	Time → 1	2	3	4	5		
I1	IF	ID	EX	MEM	WB		
I2		IF	ID	EX	MEM	WB	
I3			IF	ID	EX	MEM	WB
I4				IF	EX	MEM	WB

(b) Pipelined Execution

Multiple instructions are in different stages together.

Instruction	Time (Clock Cycles) → 1	2	3	4	5	6	7	8
I1	IF	ID	EX	MEM	WB			
I2		IF	ID	EX	MEM	WB		
I3			IF	ID	EX	MEM	WB	
I4				IF	ID	EX	MEM	WB

Once the pipeline is full (after n cycles for n stages), one instruction completes every cycle!

4. PERFORMANCE IMPROVEMENT

Let,
k = number of stages in pipeline
t = time for each stage
n = number of instructions

- Non-Pipelined Time = $n \times k \times t$
- Pipelined Time = $(k - 1 + n) \times t$

$$\text{Speedup} = \frac{n \times k}{k + n - 1} \approx k \quad \text{(for large } n\text{)}$$

5. ADVANTAGES AND DISADVANTAGES

Advantages	Disadvantages
• Increases instruction throughput (better performance)	• More complex processor design
• Higher CPU utilization	• Pipeline hazards reduce performance
• Efficient hardware usage	• Requires more registers (pipeline registers between stages)

6. PIPELINE HAZARDS

Hazard Type	Description	Example	Solution
Structural Hazard	Two stages need the same hardware at the same time.	Single memory used for IF and MEM.	Use separate instruction and data memory.
Data Hazard	Instruction depends on result of previous instruction.	ADD R1, R2, R3 SUB R4, R1, R5	Forwarding, Stalling (Bubbles)
Control Hazard	Next instruction depends on branch/jump outcome.	BEQ R1, R2, Label	Branch Prediction, Stalling, Flush

7. PIPELINE HAZARD EXAMPLE

Data Hazard (RAW – Read After Write)

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
```
↔ I2 needs R1 result from I1

Solution:
- Forward the result from EX/MEM stage of I1 to EX stage of I2.
- If not possible, insert a stall (bubble).

8. IMPORTANT POINTS

★ Pipelining improves throughput, not latency.
★ Ideal speedup is equal to the number of stages (k).
★ Hazards are the main reason for performance loss in pipelines.
★ Modern CPUs use advanced techniques like forwarding, branch prediction and out-of-order execution to reduce hazards.

KEY TAKEAWAY

Pipelining breaks instruction execution into smaller steps.
Multiple instructions are processed simultaneously in different stages, greatly improving the CPU performance.
Efficient handling of hazards is the key to a high-performance pipeline.

Think: Pipelining is like a water pipeline – once filled, it keeps delivering output continuously!

12. RISC vs CISC

RISC and CISC are two different approaches to instruction set design in computer architecture. They trade off between simplicity and performance.

Feature	RISC (Reduced Instruction Set Computer)	vs	CISC (Complex Instruction Set Computer)
Full Form	Reduced Instruction Set Computer		Complex Instruction Set Computer
Design Philosophy	Use simple instructions that take 1 clock cycle. Focus on speed and efficiency.		Use complex instructions that may take multiple clock cycles. Focus on reducing code size.
Instruction Set	Small and simple		Large and complex
Instruction Length	Fixed length (e.g., 32 bits)		Variable length
Addressing Modes	Few (Limited)		Many (Extensive)
Number of Instructions	Less		More
Execution Time	Most instructions execute in 1 clock cycle		Instructions may take multiple clock cycles
Control Unit	Simple		Complex (microprogrammed)
Hardware Complexity	Less		More
Registers	Large number of general purpose registers		Fewer general purpose registers
Memory Access	Load/Store architecture (Only LOAD and STORE access memory)		Memory can be accessed by many instructions (e.g., ADD [memory], JUMP [memory])
Compiler	Requires more instructions, but simpler to optimize		Requires fewer instructions, but harder to optimize
Code Size	Larger		Smaller
Performance	Higher (more instructions per second)		Lower (fewer instructions per second)
Examples	ARM, MIPS, SPARC, RISC-V, PowerPC		x86, x86-64, 68000, VAX

RISC vs CISC: Execution Example

Expression: A = B + C * D

RISC Approach (Simple Instructions)

```
1. LOAD R1, B        ; R1 = B
2. LOAD R2, C        ; R2 = C
3. LOAD R3, D        ; R3 = D
4. MUL  R2, R2, R3   ; R2 = C * D
5. ADD  R1, R1, R2   ; R1 = B + (C*D)
6. STORE R1, A       ; A = R1
```

Total: 6 instructions

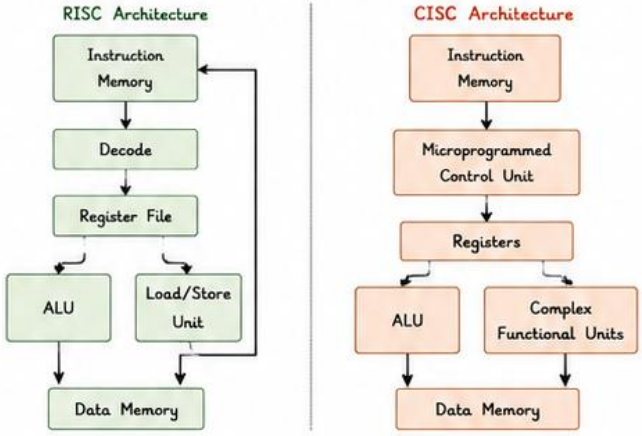
CISC Approach (Complex Instructions)

```
1. MOV  AX, B
2. MUL  C, D         ; AX = C * D
3. ADD  AX, B        ; AX = B + (C*D)
4. MOV  A, AX
```

Total: 4 instructions

CISC uses fewer instructions to perform the same task, but each instruction may take more than one clock cycle.

RISC vs CISC: Structural Comparison



Advantages of RISC

- Simple instructions allow faster execution.
- Easier to design and implement.
- Higher performance and better pipelining.
- More registers → fewer memory accesses.
- Lower power consumption (in general).

Disadvantages of RISC

- Requires more instructions for complex tasks.
- Larger code size.
- More memory bandwidth needed.

Advantages of CISC

- Fewer instructions for complex tasks.
- Smaller code size.
- Better for memory-constrained systems.
- Complex addressing modes reduce memory operations.

Disadvantages of CISC

- Complex instructions take more clock cycles.
- Harder tor design and implement.
- Lower performance compared to RISC.
- Higher power consumption.

RISC vs CISC: Key Summary

- ✓ RISC is simple, fast and efficient.
- ✓ CISC is complex, compact and code-efficient.
- ✓ RISC processors are used in modern mobile, embedded and high-performance systems.
- ✓ CISC processors are commonly used in general purpose desktops and servers (x86 family).



Modern Trend

Modern processors (like x86) internally use RISC-like techniques:

- Pipelining
- Superscalar execution
- Out-of-order execution
- Register renaming

So, CISC instructions are decoded into simpler micro-operations which are executed in a RISC-like core.

KEY TAKEAWAY

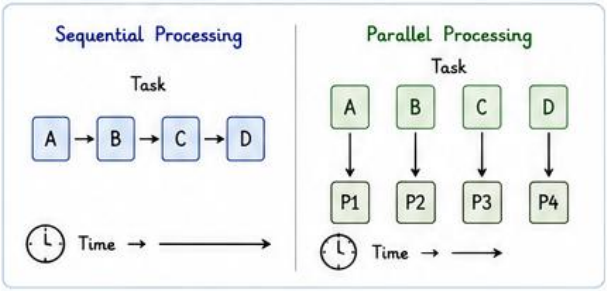
RISC focuses on doing simple things fast, while CISC focuses on doing complex things in fewer steps. The best architecture is the one that achieves maximum performance with minimum cost and power.

13. PARALLEL PROCESSING

Parallel Processing is the technique of using multiple processing units (cores/CPUs) at the same time to solve a problem faster. The problem is divided into smaller parts, executed simultaneously and combined to get the final result.

1. BASIC IDEA

- In a single processor, tasks are executed one after another (sequentially).
- In parallel processing, tasks are executed at the same time on different processors.



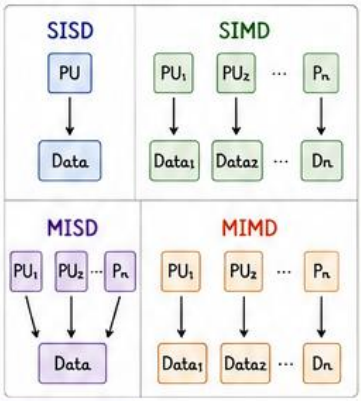
2. TYPES OF PARALLELISM

- Parallelism can be achieved at different levels.

Type	Description	Example
Bit-Level Parallelism	Operations on multiple bits of data at the same time.	32-bit adder adds 32 bits in parallel.
Instruction-Level Parallelism (ILP)	Multiple instructions are overlapped in execution.	Pipelining, Superscalar execution.
Data-Level Parallelism (DLP)	Same operation on different data items simultaneously.	Vector processors, SIMD instructions.
Task-Level Parallelism (TLP)	Different tasks are executed concurrently.	Multicore CPUs, Multi-threading, Distributed systems.

3. ARCHITECTURES FOR PARALLEL PROCESSING

- SISD – Single Instruction, Single Data (Traditional single processor system)
- SIMD – Single Instruction, Multiple Data (Same instruction on multiple data)
- MISD – Multiple Instruction, Single Data (Rarely used)
- MIMD – Multiple Instruction, Multiple Data (Most general and widely used)



PU → Processing Unit (can be a core/processor)

4. BENEFITS

- ✓ Faster execution and higher throughput
- ✓ Better resource utilization
- ✓ Can solve large and complex problems
- ✓ Improves reliability (fault tolerance)
- ✓ Scalable performance (add more processors)

5. CHALLENGES

- Communication and synchronization overhead
- More complex hardware and software design
- Load balancing is difficult
- Parallel programs are harder to write and debug
- Not all problems can be parallelized

6. EXAMPLE: VECTOR ADDITION

Sequential

Add two arrays A and B of size 8.

C[i] = A[i] + B[i] for i = 0 to 7

0 → 1 → 2 → 3 → 4 → 5 → 6 → 7

(One by one)

Parallel (4 Processors)

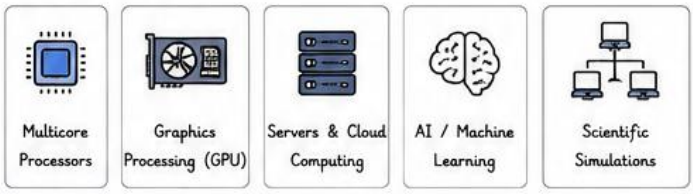
Divide the work among 4 processors.

P1 P2 P3 P4

0,1 2,3 4,5 6,7

All computed at the same time

7. APPLICATIONS



8. COMPARISON: SINGLE CORE vs MULTICORE (PARALLEL)

Feature	Single Core (Sequential)	Multicore / Parallel Processing
Execution	One task at a time	Many tasks at the same time
Performance	Lower for large tasks	Higher for large/parallelizable tasks
Resource Utilization	May be underutilized	Better utilization of CPU resources
Power Efficiency	Better for small tasks	Efficient for heavy workloads
Programming	Simple	Complex (threads, sync, communication)

9. KEY POINTS

- ★ Parallel processing uses multiple processors to work on parts of a task simultaneously.
- ★ It increases speed and efficiency.
- ★ Achieved through different types of parallelism.
- ★ MIMD is the most general and practical model.
- ★ Requires careful handling of communication and synchronization.

KEY TAKEAWAY

Parallel Processing is the heart of modern computing.
More cores + smart parallel algorithms = massive speedup!

14. IMPORTANT NUMERICAL & EXAM PROBLEMS

This section covers the most important numerical problems asked in exams on Computer Architecture.
Learn the formulas, understand the method and practice regularly!

1. PERFORMANCE CALCULATION

(a) CPU Time

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$\text{Clock Cycle Time} = 1 / \text{Clock Rate}$$

(b) MIPS (Million Instructions Per Second)

$$\text{MIPS} = \frac{\text{Instruction Count}}{\text{CPU Time} \times 10^6}$$

(c) Example

A program has 10^9 instructions.
CPI = 2, Clock Rate = 2 GHz
Find CPU Time and MIPS.

Solution:

$$\text{CPU Time} = 10^9 \times 2 \times \frac{1}{(2 \times 10^9)} = 1 \text{ second}$$

$$\text{MIPS} = \frac{10^9}{1 \times 10^6} = 1000 \text{ MIPS}$$

2. Amdahl's Law (Speedup)

$$\text{Speedup (S)} = \frac{1}{(1 - P) + P/N}$$

Where,
P = Fraction of program that can be improved
N = Number of processors
(1 − P) = Fraction that remains serial

Example:
80% of a program can be parallelized.
Find speedup using 4 processors.

P = 0.8, N = 4

S = 1 / [(1 − 0.8) + 0.8/4]
= 1 / [0.2 + 0.2]
= 1 / 0.4 = 2.5

3. Memory Access Time (With Cache)

$$\text{Average Memory Access Time (AMAT)} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

Where,
Hit Time = Time to access cache (in hit)
Miss Penalty = Time to access main memory

Example:
Hit Time = 2 ns, Miss Rate = 5%
Miss Penalty = 100 ns

AMAT = 2 + 0.05 × 100
= 2 + 5 = 7 ns

$$\text{Miss Rate} = \frac{\text{Misses}}{\text{Total Memory Accesses}}$$

4. CACHE PROBLEMS

(a) Number of Cache Lines

$$\text{Number of Lines} = \frac{\text{Cache Size}}{\text{Block Size}}$$

(b) Example

Cache size = 32 KB, Block size = 64 Bytes
Find number of cache lines.

Number of Lines = 32 × 1024 / 64 = 512 lines

(c) Address Breakdown

Cache Size	Block Size	Address Bits	Tag Bits	Index Bits	Offset Bits
32 KB	64 B	32	(32 − (9 + 6))	9	6

Offset bits = $\log_2$(Block Size) = $\log_2(64)$ = 6
Index bits = $\log_2$(Number of Lines) = $\log_2(512)$ = 9
Tag bits = 32 − (6 + 9) = 17 bits

5. DIRECT MAPPED CACHE

(a) Line Number

$$\text{Line Number} = (\text{Block Address}) \bmod (\text{Number of Lines})$$

(b) Example

A memory block with address 120 is to be stored in a cache with 16 lines. Block size = 4 words.

Block Address = 120 / 4 = 30

Line Number = 30 mod 16 = 14

(c) Hit or Miss?

If tag matches and valid bit = 1 → Hit
Else → Miss

6. VIRTUAL MEMORY

(a) Page Number and Offset

$$\text{Page Number} = \frac{\text{Virtual Address}}{\text{Page Size}}$$

$$\text{Offset} = \text{Virtual Address} \bmod \text{Page Size}$$

(b) Example

Page size = 4 KB (2^{12}), Virtual Address = 0x12345
Page Number = 0x12345 / 0x1000 = 0x12
Offset = 0x12345 mod 0x1000 = 0x345

(c) Page Table Entry Format (Example)

Valid	Protection	Reference	Frame Number
1 bit	3 bits	1 bit	m bits

7. BUS BANDWIDTH

$$\text{Bandwidth} = \frac{\text{Bus Width (bits)}}{8} \times \frac{\text{Bus Frequency (Hz)}}{10^6} \text{ (MB/s)}$$

Example:
Bus width = 64 bits, Bus frequency = 100 MHz
Bandwidth = 64/8 × 100 /10^6 = 800 MB/s

8. INSTRUCTION CYCLE TIME

$$\text{Cycle Time} = \frac{1}{\text{Clock Rate}}$$

Example:
Clock Rate = 2 GHz
Cycle Time = 1 / 2 × 10^9 = 0.5 ns

9. CPI AVERAGE

$$\text{Average CPI} = \frac{\sum (\text{CPI}_i \times \text{Instruction}_i)}{\text{Total Instructions}}$$

Example:
Type A: 2 CPI, 40% instructions
Type B: 1 CPI, 60% instructions
Average CPI = (2 × 0.4) + (1 × 0.6) = 1.4

10. IMPORTANT SHORT QUESTIONS (WITH ANSWERS)

Q. No.	Question	Short Answer
1	What is MIPS?	Million Instructions Per Second. It measures how many million instructions a CPU executes in one second.
2	What is CPI?	Cycles Per Instruction. Average number of clock cycles required to execute one instruction.
3	What is the advantage of cache?	It reduces average memory access time by storing frequently used data closer to CPU.
4	What is the purpose of virtual memory?	It allows the execution of larger programs by using secondary memory as an extension of main memory.
5	What is pipelining?	It is the technique of overlapping instruction execution in different stages to increase throughput.

FORMULA QUICK REFERENCE

1. CPU Time = IC × CPI × Cycle Time
2. MIPS = IC / (CPU Time × 10^6)
3. Speedup (Amdahl) = 1 / (1 − P) + P/N)
4. AMAT = Hit Time + Miss Rate × Miss Penalty
5. #Lines = Cache Size / Block Size
6. Bandwidth = (Bus Width/8) × (Bus Frequency/10^6)
7. Cycle Time = 1 / Clock Rate
8. Average CPI = $\sum$(CPI_i × Inst_i) / Total Inst

KEY TAKEAWAY

★ Understand the concept first, then remember the formula.
★ Practice numerical problems regularly.
★ In exams, write steps clearly and show calculations.
★ These problems are high scoring – don't skip!

15. INTERVIEW QUESTIONS + QUICK REVISION CHEAT SHEET

Revise Smart Score High!

Practice these interview questions to build strong concepts and confidence. Keep the cheat sheet handy for quick revision before exams and interviews!

A. IMPORTANT INTERVIEW QUESTIONS (with short points)

1 What is the difference between RISC and CISC?

- RISC: Simple instructions, fixed length, load/store architecture, more registers, easier pipelining.
- CISC: Complex instructions, variable length, memory-to-memory operations, fewer registers.

2 What is pipelining? What are its advantages and hazards?

- Pipelining divides instruction execution into stages and overlaps them to increase throughput.
- Advantages: Higher performance, better CPU utilization.
- Hazards: Structural, Data, Control.

3 What is cache memory? Why is it used?

- Cache is a small, high-speed memory between CPU and main memory.
- It stores frequently used data/instructions to reduce average memory access time and improve performance.

4 What is the role of the bus in a computer system?

- Bus is the communication pathway that transfers data, address and control signals between CPU, memory and I/O devices.
- Types: Data Bus, Address Bus, Control Bus.

5 What is the difference between SRAM and DRAM?

- SRAM: Faster, no refresh needed, expensive, used for cache.
- DRAM: Slower, needs periodic refresh, cheaper, used for main memory.

6 What is virtual memory? How does it work?

- Virtual memory allows execution of larger programs than main memory by using secondary memory as an extension.
- Uses paging/segmentation and page table for address translation.

7 What is the instruction cycle?

- Fetch → Decode → Execute → Memory Access → Write Back.
- This cycle repeats for every instruction.

8 What are addressing modes? Name a few.

- They specify how the operand is located.
- Types: Immediate, Register, Direct, Indirect, Register Indirect, Indexed, Base + Displacement.

9 What is parallel processing? Where is it used?

- Using multiple processors/cores simultaneously to solve a problem.
- Used in CPUs (multicore), GPUs, supercomputers, AI/ML, scientific simulations.

10 How can cache performance be improved?

- Increase cache size, reduce miss rate, use better replacement policy (LRU), increase associativity.

Pro Tip: While answering, give real-time examples (CPU, mobile, servers), diagrams if needed, and keep answers crisp and structured.

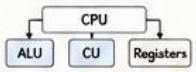
B. QUICK REVISION CHEAT SHEET

1. Computer Architecture

- Study of structure, behavior and implementation of computer systems.
- Focus: CPU, Memory, I/O and their interaction.

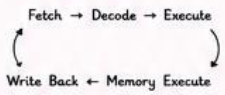
2. CPU Organization

- ALU: Performs arithmetic & logic operations.
- CU: Controls all operations.
- Registers: Store data, instructions, addresses.



3. Instruction Cycle

1. Fetch (Get instruction)
2. Decode (Understand)
3. Execute (Perform)
4. Memory Access
5. Write Back



4. Instruction Formats

Fields: Opcode | Addressing Mode | Operand/Address

Types: Data Transfer, Arithmetic, Logical, Branch, I/O, System.

Opcode	Mode	Address/ Operand

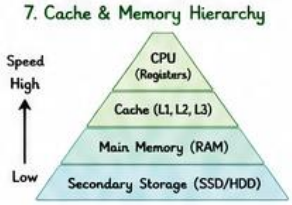
5. Addressing Modes

- Immediate
- Register
- Direct
- Indirect
- Register Indirect
- Indexed
- Base + Displacement

6. Memory Organization

- Main memory (RAM)
- ROM / Non-volatile memory
- Organization depends on word length, address space, data width.
- Types: SRAM, DRAM, ROM, Flash, EEPROM.

7. Cache & Memory Hierarchy



- Higher level = faster & smaller
- Lower level = slower & larger
- Goal: Minimize average memory access time

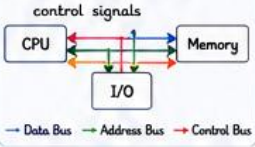
8. I/O Organization

- I/O devices connect via interfaces/controllers.
- Types: Programmed I/O, Interrupt-driven I/O, DMA.
- Data transfer: CPU ↔ I/O Module ↔ Device.

9. System Bus

Three main buses:

- **Data Bus**: Carries data
- **Address Bus**: Carries address
- **Control Bus**: Carries control signals



10. Pipelining

- Divide instruction execution into stages.
- Overlap stages to improve throughput.
- 5 Stages: IF, ID, EX, MEM, WB.

	Time → 1	2	3	4	5	7
I1	IF	ID	EX	MEM	WB	
I2		IF	ID	EX	MEM	
I3			IF	ID	EX	MEM

11. Parallel Processing

- Multiple processors/cores work ismultaneously.
- Types: SISD, SIMD, MISD, MIMD.
- Increases speed and CPU utilization.

12. RISC vs CISC

Feature	RISC	CISC
Instruction	Simple	Complex
Length	Fixed	Variable
Addressing	Few	Many
Registers	More	Fewer
Cycles/Inst	1 (often)	Multiple
Pipelining	Easy	Difficult
Examples	ARM, RISC-V	x86, 68000

13. Performance Formulas

- CPU Time = IC × CPI × Clock Cycle Time
- MIPS = IC / (CPU Time × 10^6)
- Speedup (Amdahl) = $\frac{1}{(1 - P) + P/N}$
- AMAT = Hit Time + Miss Rate × Miss Penalty
- CPI (avg) = $\frac{\Sigma (CPI_i \times I_i)}{\Sigma I_i}$
- Bus Bandwidth = $\frac{\text{Bus Width}}{8} \times \frac{\text{Bus Freq}}{10^6}$

14. Common Units

- ALU – Arithmetic & Logic
- CU – Control Unit
- Registers – Fast storage
- PC – Program Counter
- MAR – Memory Address Reg.
- MDR – Memory Data Reg.
- IR – Instruction Register

15. Important Points

- ✓ Higher cache hit rate → better performance.
- ✓ Pipelining improves throughput, not latency.
- ✓ Parallelism uses multiple cores.
- ✓ Good locality improves cache performance.
- ✓ Buses are shared; arbitration avoids conflicts.

Remember: Understand concepts → Practice numericals → Draw diagrams → Revise regularly.
That's the formula for scoring high in exams & cracking interviews!